Implementation of Boosting Techniques (AdaBoost) and Analyze Performance

Aim

To implement the **AdaBoost** boosting technique, which combines several weak learners (decision stumps) into a strong classifier, and to analyze its performance using **accuracy, precision, recall, F1-score, ROC-AUC** and **cross-validation**.

Algorithm

A. AdaBoost Implementation (binary, labels y ∈ {−1, +1})

Input: Training set $(x_1, y_1), \ldots, (x_N, y_N)$; number of rounds **M**; weak learner (decision stump)

Step 1: Initialise the sample weights: $\mathbf{w_i = 1/N}$ for all i.

Step 2: For **m = 1 to M**:

1. Train the weak learner $\mathbf{h_m(x)}$ on the training data using the weights w.
2. Compute the weighted error:
   $\boldsymbol{\varepsilon_m = \Sigma\, w_i \cdot [h_m(x_i) \neq y_i] \,/\, \Sigma\, w_i}$
3. Compute the learner weight:
   $\boldsymbol{\alpha_m = ½ \cdot \ln((1 - \varepsilon_m) / \varepsilon_m)}$
4. Update the sample weights:
   $\boldsymbol{w_i \leftarrow w_i \cdot \exp(-\alpha_m \cdot y_i \cdot h_m(x_i))}$
   (misclassified samples receive higher weights)
5. Normalise the weights so that $\boldsymbol{\Sigma\, w_i = 1}$.

Step 3: Output the final strong classifier:
$\boldsymbol{H(x) = \text{sign}(\, \Sigma_m\, \alpha_m \cdot h_m(x)\,)}$

B. Performance Analysis Procedure

Step 1: Load the dataset and split it into **training and test sets** (stratified).

Step 2: Train a **single weak learner** (decision stump) as the baseline.

Step 3: Train the **AdaBoost** model with M estimators.

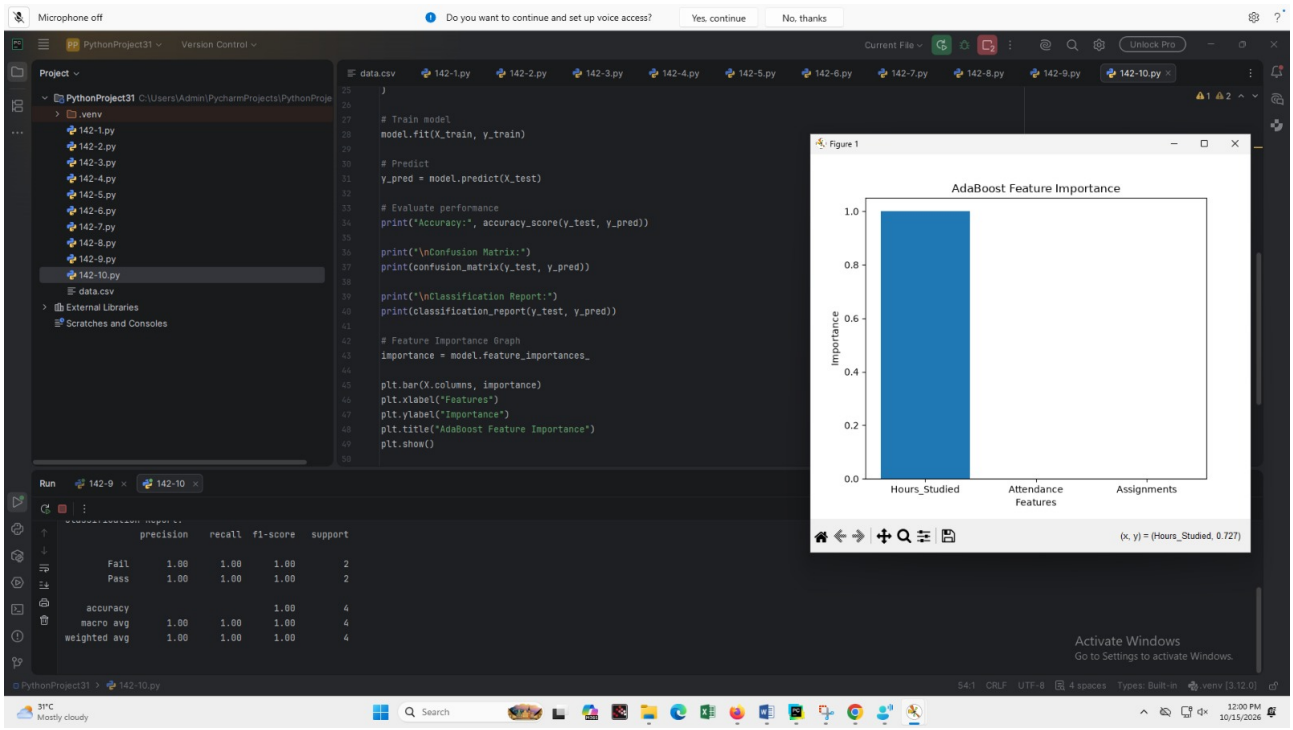
Step 4: Predict on the test set and compute **accuracy, precision, recall, F1-score, ROC-AUC** and the **confusion matrix**.

Step 5: Perform **k-fold cross-validation** and record the **training time**.

Step 6: Plot **error vs. number of boosting rounds** and the **ROC curve**.

Step 7: Compare AdaBoost with the baseline and **conclude** whether boosting improved performance.

Program


Microphone off
Do you want to continue and set up voice access?
Yes, continue
No, thanks
PythonProject31
Version Control
Current File
Unlock Pro
Project
PythonProject31 C:\Users\Admin\PycharmProjects\PythonProje
.venv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
142-6.py
142-7.py
142-8.py
142-9.py
142-10.py
data.csv
External Libraries
Scratches and Consoles
data.csv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
142-6.py
142-7.py
142-8.py
142-9.py
142-10.py
)
Train model
model.fit(X_train, y_train)
Predict
y_pred = model.predict(X_test)
Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
Feature Importance Graph
importance = model.feature_importances_
plt.bar(X.columns, importance)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.show()
Figure 1
AdaBoost Feature Importance
Importance
1.0
0.8
0.6
0.4
0.2
0.0
Hours_Studied
Attendance
Assignments
Features
(x, y) = (Hours_Studied, 0.727)
Run
142-9
142-10
precision recall f1-score support
Fail 1.00 1.00 1.00 2
Pass 1.00 1.00 1.00 2
accuracy 1.00 4
macro avg 1.00 1.00 1.00 4
weighted avg 1.00 1.00 1.00 4
Activate Windows
Go to Settings to activate Windows.
PythonProject31 > 142-10.py
54:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.12.0]
31°C
Mostly cloudy
Search
12:00 PM
10/15/2026